

Polynomial Multiplication using Number Theoretic Transform (NTT)

Abstract—Polynomial multiplication is a fundamental operation in many modern cryptographic algorithms, especially lattice-based cryptography and homomorphic encryption schemes. Traditional polynomial multiplication methods have quadratic time complexity. The Number Theoretic Transform (NTT) provides an efficient approach for performing polynomial multiplication in the modular domain by transforming the coefficients into the frequency domain. This paper presents an implementation of polynomial multiplication using the NTT algorithm. The proposed approach leverages modular arithmetic and butterfly operations to achieve fast polynomial convolution with complexity of $O(n \log n)$. The architecture is suitable for hardware acceleration and applications in post-quantum cryptographic systems.

Index Terms—Polynomial Multiplication, Number Theoretic Transform, Post-Quantum Cryptography, Modular Arithmetic, Hardware Acceleration

I. INTRODUCTION

Polynomial multiplication is widely used in signal processing, coding theory, and modern cryptography. In particular, lattice-based cryptographic schemes such as Kyber, NTRU, and homomorphic encryption rely heavily on efficient polynomial multiplication operations.

The naive polynomial multiplication method has a time complexity of $O(n^2)$, which becomes computationally expensive for large polynomial degrees. The Number Theoretic Transform (NTT) is an integer-domain analogue of the Fast Fourier Transform (FFT) and allows polynomial multiplication to be performed efficiently in modular arithmetic.

In this paper, we clearly explain how does the NTT algorithm works and how does one can implement polynomial multiplication using NTT.

II. BACKGROUND

A. Polynomial Representation

Let two polynomials be defined as

$$A(x) = \sum_{i=0}^{n-1} a_i x^i \quad (1)$$

$$B(x) = \sum_{i=0}^{n-1} b_i x^i \quad (2)$$

The product polynomial is

$$C(x) = A(x) \cdot B(x) \quad (3)$$

The naive polynomial multiplication requires $O(n^2)$ multiplications, which becomes computationally expensive for large values of n .

B. Number Theoretic Transform

The Number Theoretic Transform (NTT) is the modular arithmetic analogue of the Fast Fourier Transform (FFT). It allows polynomial multiplication to be computed efficiently in the transform domain.

The forward NTT of a sequence $a = (a_0, a_1, \dots, a_{n-1})$ is defined as

$$A_k = \sum_{j=0}^{n-1} a_j \omega^{jk} \pmod{q} \quad (4)$$

where

- q is a prime modulus,
- ω is an n -th primitive root of unity modulo q such that $\omega^n \equiv 1 \pmod{q}$.

Direct computation of the NTT requires $O(n^2)$ operations. However, similar to the FFT algorithm, the NTT can be computed efficiently using recursive decompositions based on butterfly operations.

C. Cooley–Tukey Butterfly for Forward NTT

The forward NTT is commonly implemented using the Cooley–Tukey (CT) butterfly structure. The transform is recursively decomposed into smaller transforms by separating even and odd indexed coefficients.

For two inputs a and b , the CT butterfly computes

$$A = a + \omega b \pmod{q} \quad (5)$$

$$B = a - \omega b \pmod{q} \quad (6)$$

where ω is the corresponding twiddle factor. This operation combines two intermediate transform values and produces two outputs.

By recursively applying this butterfly operation across $\log_2(n)$ stages, the complete NTT can be computed with time complexity

$$O(n \log n). \quad (7)$$

The Cooley–Tukey butterfly forms the fundamental building block for efficient hardware and software implementations of the forward NTT.

D. Gentleman–Sande Butterfly for Inverse NTT

The inverse NTT is commonly implemented using the *Gentleman–Sande (GS) butterfly*, which can be interpreted as the reverse structure of the Cooley–Tukey network.

For two inputs a and b , the GS butterfly computes

$$A = a + b \pmod{q} \quad (8)$$

$$B = (a - b)\omega^{-1} \pmod{q} \quad (9)$$

where ω^{-1} is the inverse twiddle factor.

This structure redistributes the frequency-domain values produced by the forward transform and gradually reconstructs the original polynomial coefficients.

Similar to the forward transform, the inverse NTT can be computed using $\log_2(n)$ stages of GS butterflies, leading to an overall complexity of

$$O(n \log n). \quad (10)$$

III. NTT-BASED POLYNOMIAL MULTIPLICATION

Using the NTT, polynomial multiplication can be performed efficiently through three main steps:

- 1) Compute the forward NTT of both input polynomials using Cooley–Tukey butterfly operations.
- 2) Perform element-wise multiplication of the transformed coefficients.
- 3) Apply the inverse NTT using Gentleman–Sande butterfly operations to recover the final polynomial coefficients.

Mathematically, the convolution can be written as

$$C = INTT(NTT(A) \circ NTT(B)) \quad (11)$$

where $\circ$ denotes point-wise multiplication.

IV. ALGORITHM

Algorithm 1 NTT-Based Polynomial Multiplication

```

1: procedure POLYNOMIALMULTIPLY( $A, B$ )
2:    $A_{NTT} \leftarrow NTT_{CT}(A)$ 
3:    $B_{NTT} \leftarrow NTT_{CT}(B)$ 
4:   for  $i = 0$  to  $n - 1$  do
5:      $C_i \leftarrow A_{NTT}[i] \times B_{NTT}[i]$ 
6:   end for
7:    $C \leftarrow INTT_{GS}(C)$ 
8:   return  $C$ 
9: end procedure

```

V. MATHEMATICAL PROOFS

A. Proof that $INTT(NTT(f)) = f$

Let the polynomial be represented by the coefficient vector

$$f = (f_0, f_1, \dots, f_{n-1}) \quad (12)$$

The Number Theoretic Transform (NTT) of f is defined as

$$\hat{f}_k = \sum_{j=0}^{n-1} f_j \omega^{jk} \pmod{q} \quad (13)$$

where ω is an n -th primitive root of unity modulo q such that

$$\omega^n \equiv 1 \pmod{q} \quad (14)$$

The inverse NTT is defined as

$$f_k = n^{-1} \sum_{\ell=0}^{n-1} \hat{f}_\ell \omega^{-k\ell} \pmod{q} \quad (15)$$

Substituting the definition of $\hat{f}_\ell$ into the inverse transform:

$$INTT(NTT(f))_k = n^{-1} \sum_{\ell=0}^{n-1} \left(\sum_{j=0}^{n-1} f_j \omega^{j\ell} \right) \omega^{-k\ell} \quad (16)$$

Rearranging the summations:

$$INTT(NTT(f))_k = n^{-1} \sum_{j=0}^{n-1} f_j \sum_{\ell=0}^{n-1} \omega^{\ell(j-k)} \quad (17)$$

Now consider the inner summation

$$S = \sum_{\ell=0}^{n-1} \omega^{\ell(j-k)} \quad (18)$$

Two cases arise.

Case 1: $j = k$

$$S = \sum_{\ell=0}^{n-1} 1 = n \quad (19)$$

Thus

$$n^{-1} f_k \cdot n = f_k \quad (20)$$

Case 2: $j \neq k$

Let $c = j - k$. Then

$$S = \sum_{\ell=0}^{n-1} \omega^{\ell c} \quad (21)$$

This is a geometric series:

$$S = \frac{1 - \omega^{nc}}{1 - \omega^c} \quad (22)$$

Since $\omega^n \equiv 1 \pmod{q}$,

$$\omega^{nc} \equiv 1 \quad (23)$$

Thus

$$S = 0 \quad (24)$$

Therefore all terms where $j \neq k$ vanish.

Hence

$$INTT(NTT(f))_k = f_k \quad (25)$$

which proves

$$INTT(NTT(f)) = f \quad (26)$$

This shows that the inverse Number Theoretic Transform perfectly recovers the original polynomial coefficients.

B. Proof that $NTT(F \cdot G) = NTT(F) \circ NTT(G)$

Let two polynomials be defined as

$$F(x) = \sum_{i=0}^{n-1} f_i x^i \quad (27)$$

$$G(x) = \sum_{j=0}^{n-1} g_j x^j \quad (28)$$

Their product polynomial is

$$H(x) = F(x)G(x) \quad (29)$$

Expanding the multiplication,

$$H(x) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} f_i g_j x^{i+j} \quad (30)$$

When working in the ring $\mathbb{Z}_q[x]/(x^n - 1)$, the exponents are reduced modulo n . Therefore the coefficients of the resulting polynomial are

$$h_k = \sum_{(i+j) \equiv k \pmod{n}} f_i g_j \quad (31)$$

Thus the polynomial multiplication corresponds to a cyclic convolution.

Now consider the NTT of H :

$$NTT(H)_k = \sum_{t=0}^{n-1} h_t \omega^{tk} \quad (32)$$

Substituting the expression for h_t ,

$$NTT(H)_k = \sum_{t=0}^{n-1} \left(\sum_{i+j \equiv t \pmod{n}} f_i g_j \right) \omega^{tk} \quad (33)$$

Rearranging the summations gives

$$NTT(H)_k = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} f_i g_j \omega^{(i+j)k} \quad (34)$$

Using the property of exponents,

$$\omega^{(i+j)k} = \omega^{ik} \omega^{jk} \quad (35)$$

Thus

$$NTT(H)_k = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} f_i g_j \omega^{ik} \omega^{jk} \quad (36)$$

Separating the sums,

$$NTT(H)_k = \left(\sum_{i=0}^{n-1} f_i \omega^{ik} \right) \left(\sum_{j=0}^{n-1} g_j \omega^{jk} \right) \quad (37)$$

Recognizing these expressions as the NTT of F and G ,

$$NTT(F)_k = \sum_{i=0}^{n-1} f_i \omega^{ik} \quad (38)$$

$$NTT(G)_k = \sum_{j=0}^{n-1} g_j \omega^{jk} \quad (39)$$

we obtain

$$NTT(H)_k = NTT(F)_k \cdot NTT(G)_k \quad (40)$$

Therefore,

$$NTT(F \cdot G) = NTT(F) \circ NTT(G) \quad (41)$$

where $\circ$ denotes element-wise (pointwise) multiplication.

This establishes that polynomial multiplication in the coefficient domain corresponds to pointwise multiplication in the NTT domain.

C. Proof that $NTT(F) = NTT_{CT}(F)$ (Cooley–Tukey Decomposition)

The Number Theoretic Transform (NTT) of a sequence $f = (f_0, f_1, \dots, f_{n-1})$ is defined as

$$F_k = \sum_{j=0}^{n-1} f_j \omega^{jk} \pmod{q} \quad (42)$$

where ω is an n -th primitive root of unity such that

$$\omega^n \equiv 1 \pmod{q}. \quad (43)$$

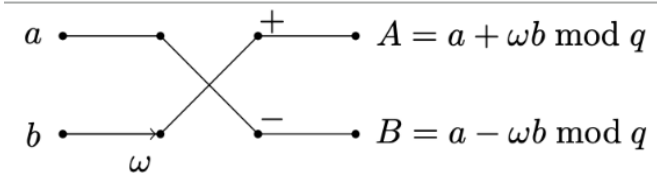
1) Even–Odd Decomposition: To derive the Cooley–Tukey formulation, the summation is split into even and odd indexed coefficients.

$$F_k = \sum_{j=0}^{n-1} f_j \omega^{jk} \quad (44)$$

$$= \sum_{j=0}^{\frac{n}{2}-1} f_{2j} \omega^{(2j)k} + \sum_{j=0}^{\frac{n}{2}-1} f_{2j+1} \omega^{(2j+1)k} \quad (45)$$

Using the identity

$$\omega^{2jk} = (\omega^2)^{jk} \quad (46)$$



we obtain

$$F_k = \sum_{j=0}^{\frac{n}{2}-1} f_{2j}(\omega^2)^{jk} + \omega^k \sum_{j=0}^{\frac{n}{2}-1} f_{2j+1}(\omega^2)^{jk} \quad (47)$$

Define the even and odd subsequences

$$f_j^{even} = f_{2j} \quad (48)$$

$$f_j^{odd} = f_{2j+1} \quad (49)$$

Then

$$F_k = NTT_{\omega^2}(f^{even})_k + \omega^k NTT_{\omega^2}(f^{odd})_k \quad (50)$$

This gives the first half of the transform outputs.

2) *Second Half of the Transform:* Using the property

$$\omega^{k+n/2} = -\omega^k \quad (51)$$

we obtain

$$F_{k+n/2} = NTT_{\omega^2}(f^{even})_k - \omega^k NTT_{\omega^2}(f^{odd})_k \quad (52)$$

Thus the transform outputs can be computed as

$$F_k = E_k + \omega^k O_k \quad (53)$$

$$F_{k+n/2} = E_k - \omega^k O_k \quad (54)$$

where

$$E_k = NTT_{\omega^2}(f^{even})_k \quad (55)$$

$$O_k = NTT_{\omega^2}(f^{odd})_k \quad (56)$$

3) *Butterfly Operation:* The above equations correspond exactly to the *Cooley–Tukey butterfly operation*:

$$A = a + \omega b \pmod{q} \quad (57)$$

$$B = a - \omega b \pmod{q} \quad (58)$$

where $a = E_k$ and $b = O_k$.

This operation combines two smaller transforms into a larger one. By recursively applying this decomposition, an n -point NTT can be computed using $\log_2 n$ stages of butterfly operations.

4) *Computational Advantage:* The direct computation of the NTT requires

$$O(n^2) \quad (59)$$

operations.

Using the Cooley–Tukey decomposition, the transform is recursively split into smaller transforms of size $n/2$, leading to the recurrence

$$T(n) = 2T(n/2) + O(n) \quad (60)$$

which solves to

$$T(n) = O(n \log n). \quad (61)$$

Therefore, the Cooley–Tukey butterfly structure computes exactly the same transform as the mathematical NTT definition but with significantly lower computational complexity.

Hence

$$NTT(F) = NTT_{CT}(F). \quad (62)$$

This recursive decomposition is identical to the Cooley–Tukey FFT algorithm used for the Fast Fourier Transform, but performed over a finite field using modular arithmetic.

D. *Proof that $INTT(F) = INTT_{GS}(F)$ (Gentleman–Sande Decomposition)*

The inverse Number Theoretic Transform (INTT) of a vector $\hat{f} = (\hat{f}_0, \hat{f}_1, \dots, \hat{f}_{n-1})$ is defined as

$$f_k = n^{-1} \sum_{j=0}^{n-1} \hat{f}_j \omega^{-jk} \pmod{q} \quad (63)$$

where ω is an n -th primitive root of unity such that

$$\omega^n \equiv 1 \pmod{q}. \quad (64)$$

1) *Decimation-in-Frequency Decomposition:* To derive the Gentleman–Sande structure, we split the sum into two halves.

$$f_k = n^{-1} \left(\sum_{j=0}^{\frac{n}{2}-1} \hat{f}_j \omega^{-jk} + \sum_{j=\frac{n}{2}}^{n-1} \hat{f}_j \omega^{-jk} \right) \quad (65)$$

Let

$$j = r + \frac{n}{2} \quad (66)$$

in the second summation.

$$f_k = n^{-1} \left(\sum_{r=0}^{\frac{n}{2}-1} \hat{f}_r \omega^{-rk} + \sum_{r=0}^{\frac{n}{2}-1} \hat{f}_{r+n/2} \omega^{-(r+n/2)k} \right) \quad (67)$$

Using the identity

$$\omega^{-nk/2} = (-1)^k \quad (68)$$

we obtain

$$f_k = n^{-1} \sum_{r=0}^{\frac{n}{2}-1} \left(\hat{f}_r + (-1)^k \hat{f}_{r+n/2} \right) \omega^{-rk}. \quad (69)$$

Define

$$A_r = \hat{f}_r + \hat{f}_{r+n/2} \quad (70)$$

$$B_r = (\hat{f}_r - \hat{f}_{r+n/2}) \omega^{-r} \quad (71)$$

Then the transform becomes

$$f_k = n^{-1} \sum_{r=0}^{\frac{n}{2}-1} A_r \omega^{-rk} \quad \text{for even } k \quad (72)$$

$$f_k = n^{-1} \sum_{r=0}^{\frac{n}{2}-1} B_r \omega^{-rk} \quad \text{for odd } k. \quad (73)$$

2) *Gentleman–Sande Butterfly*: The above decomposition corresponds to the *Gentleman–Sande butterfly operation*

$$A = a + b \pmod{q} \quad (74)$$

$$B = (a - b) \omega^{-k} \pmod{q} \quad (75)$$

where $a = \hat{f}_r$ and $b = \hat{f}_{r+n/2}$.

This butterfly combines two frequency-domain values and distributes them into smaller inverse transforms.

3) *Recursive Structure*: Using this decomposition, the inverse transform of size n is reduced into two transforms of size $n/2$:

$$T(n) = 2T(n/2) + O(n) \quad (76)$$

which results in the overall complexity

$$T(n) = O(n \log n). \quad (77)$$

4) *Relationship with Cooley–Tukey*: The Gentleman–Sande butterfly can be interpreted as the reverse of the Cooley–Tukey butterfly network. While the forward NTT uses the CT structure to mix the coefficients, the inverse transform applies the GS structure to undo this mixing and recover the original coefficients.

Therefore the inverse transform can be implemented using the Gentleman–Sande butterfly network, giving

$$INTT(F) = INTT_{GS}(F). \quad (78)$$